Assign 3

과제 풀이 정리 · Notion 아카이브

UCS - `frontier.update(state,cost)` | `frontier.removeMin()`

```
class UniformCostSearch(SearchAlgorithm):
    def __init__(self, verbose: int = 0):
        self.verbose = verbose
    def solve(self, problem: SearchProblem, heuristic=lambda x: 0):
        self.actions = None
        self.totalCost = None
        self.numStatesExplored = 0
        frontier = priorityQueue()
        backpointers = {}
        startState = problem.startState()
        frontier.update(startState, 0)
        while True:
            state, pastCost = frontier.removeMin()
            if state is None:
                break
            self.numStatesExplored += 1
            if problem.isEnd(state):
                goalState = state
                self.actions = []
                while state != startState:
                    action, prevState = backpointers[state]
                    self.actions.append(action)
                    state = prevState
                self.actions.reverse()
                self.totalCost = (pastCost - heuristic(goalState) + heuristic(startState))
                return
            for action, newState, cost in problem.succAndCost(state):
                newCost = (pastCost + cost + heuristic(newState) - heuristic(state))
                if frontier.update(newState, newCost):
                    backpointers[newState] = (action, state)
```

단, 이 구현에서 휴리스틱을 사용할 때는 모든 간선에 대해 cost + h(newState) - h(state) >= 0이어야 합니다. 즉, 휴리스틱의 일관성이 필요

Problem 1 : *Language Models and Search Cost 1*

1a÷Word Segmentation(단어 나누기)

띄어쓰기가 없는 string 입력받고

-> 그럴듯한 곳에 띄어쓰기를 해야하는 문제

```
================================================================
class WordSegmentationProblem(util.SearchProblem):
    def __init__(self, query: str, unigramCost: Callable[[str], float]):
        self.query = query
        self.unigramCost = unigramCost
    def startState(self) -> int:
        return 0
    def isEnd(self, state: int) -> bool:
        return state == len(self.query)
    def succAndCost(self, state: int) -> Iterable[Tuple[str, int, float]]:
        succAndCost_list = []
        for end_idx in range(state + 1, len(self.query) + 1):
            succ = self.query[state:end_idx]
            cost = self.unigramCost(succ)
            succAndCost_list.append((succ, end_idx, cost))
        return succAndCost_list
================================================================
def segmentWords(
    query: str,
    unigramCost: Callable[[str], float]
) -> str:
    ucs = util.UniformCostSearch(verbose=0)
    ucs.solve(WordSegmentationProblem(query, unigramCost))
    if ucs.actions is None:
        return ""
    return " ".join(ucs.actions)
```

1b÷Exact-k Word Segmentation

띄어쓰기가 없는 string + k

-> k개의 단어로 만듦

```
================================================================
class ExactKWordSegmentationProblem(util.SearchProblem):
    def __init__(self, query: str, k: int, unigramCost: Callable[[str], float]):
        self.query = query
```

```
        self.k = k
        self.unigramCost = unigramCost
    def startState(self) -> Tuple[int, int]:
        return (0, 0)
    def isEnd(self, state: Tuple[int, int]) -> bool:
        return (state[0] == len(self.query) and state[1] == self.k)
    def succAndCost(self, state: Tuple[int, int]) -> Iterable[Tuple[str, Tuple[int, int], float]]:
        if state[1] == self.k:
            return []
        succAndCost_list = []
        for end_idx in range(state[0] + 1, len(self.query) + 1):
            succ = self.query[state[0]:end_idx]
            cost = self.unigramCost(succ)
            succAndCost_list.append((succ, (end_idx, state[1] + 1), cost))
        return succAndCost_list
============================================================
def segmentExactKWords(
    query: str,
    k: int,
    unigramCost: Callable[[str], float]
) -> str:
    if len(query) == 0 or k <= 0 or k > len(query):
        return ""
    ucs = util.UniformCostSearch(verbose=0)
    problem = ExactKWordSegmentationProblem(
        query, k, unigramCost
    )
    ucs.solve(problem)
    if ucs.actions is None:
        return ""
    return " ".join(ucs.actions)
```

Problem 2 : *Language Models and Search Cost 2*

2a÷Vowel Insertion(모음 삽입)

```
class VowelInsertionProblem(util.SearchProblem):
    def __init__(
        self,
        queryWords: List[str],
```

```
        bigramCost: Callable[[str, str], float],
        possibleFills: Callable[[str], Set[str]]
    ):
        self.queryWords = queryWords
        self.bigramCost = bigramCost
        self.possibleFills = possibleFills
    def startState(self) -> Tuple[int, str]:
        return (0, wordsegUtil.SENTENCE_BEGIN)
    def isEnd(self, state: Tuple[int, str]) -> bool:
        return state[0] == len(self.queryWords)
    def succAndCost(
        self,
        state: Tuple[int, str]
    ) -> Iterable[Tuple[str, Tuple[int, str], float]]:
        query_word = self.queryWords[state[0]]
        candidates = self.possibleFills(query_word)
        if not candidates:
            candidates = {query_word}
        succAndCost_list = []
        for succ in sorted(candidates):
            cost = self.bigramCost(state[1], succ)
            succAndCost_list.append(
                (succ, (state[0] + 1, succ), cost)
            )
        return succAndCost_list
============================================================
def insertVowels(
    queryWords: List[str],
    bigramCost: Callable[[str, str], float],
    possibleFills: Callable[[str], Set[str]],
) -> str:
    if len(queryWords) == 0:
        return ""
    ucs = util.UniformCostSearch(verbose=0)
    problem = VowelInsertionProblem(
        queryWords,
        bigramCost,
        possibleFills
```

```
    )
    ucs.solve(problem)
    if ucs.actions is None:
        return ""
    return " ".join(ucs.actions)
```

2b÷ Restricted-Vowel Insertion `set(c).isdisjoint(restricted_set)`

```
restrictedVowels: str -> 금지어
----------------------------------------------------------------
class RestrictedVowelInsertionProblem(util.SearchProblem):
    def __init__(
        self,
        queryWords: List[str],
        bigramCost: Callable[[str, str], float],
        possibleFills: Callable[[str], Set[str]],
        restrictedVowels: str,
    ):
        self.queryWords = queryWords
        self.bigramCost = bigramCost
        self.possibleFills = possibleFills
        self.restrictedVowels = restrictedVowels
    def startState(self) -> Tuple[int, str]:
        return (0, wordsegUtil.SENTENCE_BEGIN)
    def isEnd(self, state: Tuple[int, str]) -> bool:
        return state[0] == len(self.queryWords)
    def succAndCost(
        self,
        state: Tuple[int, str]
    ) -> Iterable[Tuple[str, Tuple[int, str], float]]:
        query_word = self.queryWords[state[0]]
        candidates = self.possibleFills(query_word)
        restricted_set = set(self.restrictedVowels)
        filtered_candidates = {
            c for c in candidates
            if set(c).isdisjoint(restricted_set)
        }
        if not filtered_candidates:
            filtered_candidates = {query_word}
```

```
        succAndCost_list = []
        for succ in sorted(filtered_candidates):
            cost = self.bigramCost(state[1], succ)
            succAndCost_list.append(
                (succ, (state[0] + 1, succ), cost)
            )
        return succAndCost_list
============================================================
def insertRestrictedVowels(
    queryWords: List[str],
    bigramCost: Callable[[str, str], float],
    possibleFills: Callable[[str], Set[str]],
    restrictedVowels: str
) -> str:
    if len(queryWords) == 0:
        return ""
    ucs = util.UniformCostSearch(verbose=0)
    problem = RestrictedVowelInsertionProblem(
        queryWords,
        bigramCost,
        possibleFills,
        restrictedVowels
    )
    ucs.solve(problem)
    if ucs.actions is None:
        return ""
    return " ".join(ucs.actions)
```

Problem 3 : *Putting It Together*

3a : Putting It Together (Joint Segmentation + Insertion)

띄어쓰기, 모음 모드 제거된 자음만 남은 문자열 -> 문으로 복구

```
------------------------------------------------------------
class JointSegmentationInsertionProblem(util.SearchProblem):
    def __init__(self, query: str, bigramCost: Callable[[str, str], float], possibleFills: Callable[[str], Set[str]]):
        self.query = query
        self.bigramCost = bigramCost
        self.possibleFills = possibleFills
    def startState(self) -> Tuple[int, str]:
```

```
        return (0, wordsegUtil.SENTENCE_BEGIN)
    def isEnd(self, state: Tuple[int, str]) -> bool:
        return state[0] == len(self.query)
    def succAndCost(self, state: Tuple[int, str]) -> Iterable[Tuple[str, Tuple[int, str], float]]:
        succAndCost_list = []
        start_idx = state[0]
        for end_idx in range(start_idx + 1, len(self.query) + 1):
            succ_set = self.possibleFills(self.query[start_idx:end_idx])
            for succ in sorted(succ_set):
                cost = self.bigramCost(state[1], succ)
                succAndCost_list.append((succ, (end_idx, succ), cost))
        return succAndCost_list
================================================================
def segmentAndInsert(
    query: str,
    bigramCost: Callable[[str, str], float],
    possibleFills: Callable[[str], Set[str]]
) -> str:
    if len(query) == 0:
        return ""
    ucs = util.UniformCostSearch(verbose=0)
    problem = JointSegmentationInsertionProblem(
        query, bigramCost, possibleFills
    )
    ucs.solve(problem)
    if ucs.actions is None:
        return ""
    return " ".join(ucs.actions)
```

3b÷ Exact-k Joint Reconstruction (k word + Restricted)

```
class ExactKJointReconstructionProblem(util.SearchProblem):
    def __init__(self, query: str, k: int, bigramCost: Callable[[str, str], float], possibleFills: Callable[[str], Set[str]]):
        self.query = query
        self.k = k
        self.bigramCost = bigramCost
        self.possibleFills = possibleFills
    def startState(self) -> Tuple[int, str, int]:
        return (0, wordsegUtil.SENTENCE_BEGIN, 0)
```

```
    def isEnd(self, state: Tuple[int, str, int]) -> bool:
        return (state[0] == len(self.query) and state[2] == self.k)
    def succAndCost(self, state: Tuple[int, str, int]) -> Iterable[Tuple[str, Tuple[int, str, int], float]]:
        if state[2] == self.k:
            return []
        succAndCost_list = []
        start_idx = state[0]
        for end_idx in range(start_idx + 1, len(self.query) + 1):
            succ_set = self.possibleFills(self.query[start_idx:end_idx])
            for succ in sorted(succ_set):
                cost = self.bigramCost(state[1], succ)
                succAndCost_list.append((succ, (end_idx, succ, state[2] + 1), cost))
        return succAndCost_list
```

return []: 현재 상태에서 가능한 successor가 없음을 의미한다.

문자열 인덱스가 매번 증가하므로 상태 그래프는 유한하며 순환하지 않는다.

따라서 탐색은 목표 상태를 찾거나 frontier가 빌 때 종료된다.

```
def segmentAndInsertExactK(
    query: str,
    k: int,
    bigramCost: Callable[[str, str], float],
    possibleFills: Callable[[str], Set[str]]
) -> str:
    if len(query) == 0 or k <= 0 or k > len(query):
        return ""
    ucs = util.UniformCostSearch(verbose=0)
    problem = ExactKJointReconstructionProblem(
        query, k, bigramCost, possibleFills
    )
    ucs.solve(problem)
    if ucs.actions is None:
        return ""
    return " ".join(ucs.actions)
```

Problem 4 : *2-Waypoint Pathfinding*

---

basic 2-way SearchProblem - `yield`

```
class MultiWaypointPathFindingProblem(util.SearchProblem):
    def __init__(self, start: tuple, waypoints: list, goal: tuple,
                 moveCost: Callable[[tuple, tuple], float],
                 possibleMoves: Callable[[tuple], Set[tuple]]):
        self.start = start
        self.waypoints = list(waypoints)  # exactly 2 waypoints: [W1, W2]
        self.goal = goal
        self.moveCost = moveCost
        self.possibleMoves = possibleMoves
        if len(self.waypoints) != 2:
            raise ValueError("Exactly two waypoints are required.")
    def startState(self):
        return (self.start, self.start == self.waypoints[0], self.start == self.waypoints[1])
    def isEnd(self, state) -> bool:
        position, visitedW1, visitedW2 = state
        return (position == self.goal and visitedW1 and visitedW2)
    def succAndCost(self, state):
        position, visitedW1, visitedW2 = state
        for direction, next_position in self.possibleMoves(position):
            new_W1 = (
                visitedW1
                or next_position == self.waypoints[0]
            )
            new_W2 = (
                visitedW2
                or next_position == self.waypoints[1]
            )
            yield (
                direction,
                (next_position, new_W1, new_W2),
                self.moveCost(position, direction)
            )
```

```
from typing import Optional
def UCSMultiWaypointPathFindingSearch(
    start: tuple,
    waypoints: list,
    goal: tuple,
```

```
    moveCost: Callable[[tuple, tuple], float],
    possibleMoves: Callable[[tuple], Set[tuple]]
) -> Optional[List[str]]:
    ucs = util.UniformCostSearch(verbose=0)
    problem = MultiWaypointPathFindingProblem(
        start, waypoints, goal, moveCost, possibleMoves
    )
    ucs.solve(problem)
    return ucs.actions
```

4a÷multiWaypointHeuristic

```
def multiWaypointHeuristic(waypoints: list, goal: tuple):
    def _multiWaypointHeuristic(state: tuple) -> float:
        position, visitedW1, visitedW2 = state
        def Manhattan_distance(pos1, pos2):
            x1, y1 = pos1
            x2, y2 = pos2
            return abs(x1 - x2) + abs(y1 - y2)
        h1 = Manhattan_distance(position, goal)
        unvisited_distance = []
        if not visitedW1:
            unvisited_distance.append(Manhattan_distance(position, waypoints[0]))
        if not visitedW2:
            unvisited_distance.append(Manhattan_distance(position, waypoints[1]))
        h2 = min(unvisited_distance, default=0)
        return max(h1, h2)
    return _multiWaypointHeuristic

// h1 -> goal까지의 heuristic
// h2 -> waypoint까지의 heuristic
---------------------------------------------------------------
def AStarMultiWaypointPathFindingSearch(
    start: tuple,
    waypoints: list,
    goal: tuple,
    moveCost: Callable[[tuple, tuple], float],
    possibleMoves: Callable[[tuple], Set[tuple]]
) -> Optional[List[str]]:
```

```
heuristic_func = multiWaypointHeuristic(waypoints, goal)
ucs = util.UniformCostSearch(verbose=0)
problem = MultiWaypointPathFindingProblem(
    start, waypoints, goal, moveCost, possibleMoves
)
ucs.solve(problem, heuristic=heuristic_func)
return ucs.actions
```

이 휴리스틱은 모든 가능한 이동에 대해 moveCost(position, direction) >= Manhattan_distance(position, next_position) 이 성립할 때 사용한다. 특히 상하좌우 인접 칸으로만 이동하고 각 이동 비용이 1 이상인 격자에서는 이 조건이 만족된다. 이때 h1은 목적지까지의 거리, h2는 아직 방문하지 않은 waypoint까지의 거리이며, max(h1, h2)도 admissible하며, 두 휴리스틱이 일관적이므로 최댓값 역시 일관적이다. 대각선 이동이나 비용이 Manhattan 이동 거리보다 작은 간선이 존재한다면 이 휴리스틱을 그대로 사용해서는 안 된다.

← 인공지능 · 포스텍 목차로